

Guide de Référence pour l'Entraînement de Modèles de Deep Learning

MLP • CNN • RNN • LSTM • GRU • Architecture Hybride CNN-LSTM

Réalisé par	Ismail HADDAOUI
Groupe	4IAD G3
Encadrant	Mme. Zineb HDILA
Année universitaire	2024 — 2025
Datasets	Wine (sklearn) · MNIST · Jena Climate

Table des matières

- | | |
|-----------|---|
| 1. | Introduction et Contexte |
| 2. | Présentation des Datasets Réels |
| 3. | Architecture des Modèles |
| 4. | Méthodologie et Configuration Expérimentale |
| 5. | Résultats Expérimentaux |
| 6. | Analyse Comparative et Discussion |
| 7. | Conclusion et Perspectives |

1. Introduction et Contexte

Le Deep Learning constitue aujourd'hui l'un des piliers les plus féconds de l'intelligence artificielle moderne. En s'appuyant sur des architectures neuronales profondes, cette discipline permet de résoudre des problèmes complexes couvrant aussi bien la vision par ordinateur, l'analyse de séries temporelles, le traitement du langage naturel que la reconnaissance d'activités humaines. L'objectif de ce projet est de fournir un guide de référence pratique et rigoureux qui démontre, à travers six architectures représentatives, comment aborder un problème d'apprentissage supervisé de bout en bout.

Le présent rapport documente l'implémentation complète de six modèles de Deep Learning : un Perceptron Multicouche (MLP) servant de baseline, un réseau convolutionnel (CNN) adapté au traitement d'images, trois architectures récurrentes (RNN, LSTM, GRU) dédiées aux séquences temporelles, et enfin une architecture hybride CNN-LSTM combinant extraction de caractéristiques locales et modélisation temporelle longue portée. Chaque modèle est entraîné sur un dataset réel et évalué selon une méthodologie rigoureuse incluant validation croisée, early stopping et calcul de métriques multiples (accuracy, F1-score macro, précision, rappel).

L'originalité de ce travail réside dans le choix de trois datasets réels couvrant trois paradigmes distincts du Deep Learning : (1) le dataset Wine de scikit-learn pour la classification tabulaire avec le MLP, (2) le célèbre dataset MNIST pour la classification d'images de chiffres manuscrits avec le CNN, et (3) le dataset Jena Climate 2009-2016 de l'Institut Max Planck pour la classification de séries temporelles multivariées avec les modèles récurrents et hybrides. Cette diversité permet de couvrir un spectre représentatif des cas d'usage industriels et académiques rencontrés en production.

1.1 Objectifs pédagogiques

Ce projet poursuit trois objectifs pédagogiques principaux. Premièrement, il vise à comprendre les fondements théoriques de chaque architecture en l'implémentant from scratch avec PyTorch, ce qui permet d'assimiler la mécanique des opérations tensorielles, de la rétropropagation et des fonctions d'activation. Deuxièmement, il cherche à établir une méthodologie expérimentale reproductible et comparable entre modèles, incluant le prétraitement, le split train/val/test, la normalisation et l'optimisation des hyperparamètres. Troisièmement, il propose une analyse critique des résultats afin de mettre en lumière les forces et faiblesses respectives de chaque architecture selon le type de données traité.

1.2 Plan du rapport

Le rapport est organisé en sept sections. La section 2 présente en détail les trois datasets utilisés, leur provenance, leurs caractéristiques statistiques et les tâches de classification associées. La section 3 décrit l'architecture technique de chaque modèle,

couche par couche, avec un accent sur les choix de conception. La section 4 expose la méthodologie expérimentale commune : split des données, hyperparamètres, fonction de perte, optimiseur et critères d'early stopping. La section 5 présente les résultats quantitatifs et qualitatifs de chaque modèle. La section 6 propose une analyse comparative transversale et une discussion des performances. La section 7 conclut sur les perspectives d'amélioration et les extensions possibles.

2. Présentation des Datasets Réels

Le choix des datasets est déterminant pour la validité expérimentale d'un projet de Deep Learning. Nous avons sélectionné trois datasets réels, reconnus dans la littérature scientifique, couvrant trois paradigmes fondamentaux : données tabulaires, images, et séries temporelles multivariées.

2.1 Wine Dataset (MLP)

Le dataset Wine, disponible dans scikit-learn, provient d'une étude chimique de vins cultivés dans une région d'Italie mais issus de trois cultivars différents. Il contient 178 échantillons décrits par 13 variables chimiques continues : alcool, acide malique, cendres, alcalinité des cendres, magnésium, phénols totaux, flavonoïdes, phénols non-flavonoïdes, proanthocyanidines, intensité de la couleur, teinte, OD280/OD315 de vins dilués, et proline. La tâche est une classification multiclasse à 3 classes (class_0, class_1, class_2) correspondant aux trois cultivars.

Ce dataset présente plusieurs caractéristiques pédagogiques intéressantes : il est de petite taille (178 échantillons), ce qui oblige à mettre en place une stratégie de régularisation rigoureuse pour éviter l'overfitting ; il comporte des features continues hétérogènes nécessitant une normalisation préalable ; et il est parfaitement équilibré entre les classes. Ces propriétés en font un excellent cas d'école pour évaluer la capacité d'un MLP à apprendre des frontières de décision non linéaires dans un espace de dimension modérée.

2.2 MNIST Dataset (CNN)

MNIST (Modified National Institute of Standards and Technology) est l'un des datasets les plus célèbres de la communauté Deep Learning. Il regroupe 70 000 images en niveaux de gris de 28×28 pixels représentant des chiffres manuscrits (0 à 9), réparties en 60 000 images d'entraînement et 10 000 images de test. Les images sont centrées et normalisées en taille, ce qui en fait un benchmark canonique pour les architectures convolutionnelles.

Pour des contraintes de temps de calcul, nous avons utilisé un sous-ensemble de 12 000 images d'entraînement et 2 000 images de test, tout en préservant la distribution des classes. Cette réduction permet un entraînement complet en quelques minutes sur CPU tout en démontrant clairement la supériorité des architectures convolutionnelles sur les MLP pour la classification d'images. Chaque image subit une normalisation par la moyenne (0.1307) et l'écart-type (0.3081) calculés sur l'ensemble du dataset.

2.3 Jena Climate Dataset (RNN/LSTM/GRU/CNN-LSTM)

Le dataset Jena Climate 2009-2016 a été collecté par l'Institut Max Planck de biogéochimie à Iéna, Allemagne. Il s'agit d'un enregistrement météorologique continu, mesuré toutes les 10 minutes, comprenant 14 variables atmosphériques : pression (mbar), température (°C), température potentielle (K), point de rosée (°C), humidité relative (%), pression de vapeur saturante (mbar), pression de vapeur actuelle (mbar), déficit de pression de vapeur (mbar), humidité spécifique (g/kg), concentration de vapeur d'eau (mmol/mol), densité de l'air (g/m³), vitesse du vent (m/s), vitesse maximale du vent (m/s) et direction du vent (°).

Le dataset original contient 420 551 enregistrements. Pour notre tâche de classification de séries temporelles, nous avons défini le problème suivant : étant donné une fenêtre de 24 pas de temps consécutifs (soit 4 heures de données au pas de 10 minutes, sous-échantillonné à 1 mesure par heure), prédire la saison météorologique à laquelle appartient cette fenêtre. Les 4 classes sont : Printemps (mars-mai), Été (juin-août), Automne (septembre-novembre) et Hiver (décembre-février). Cette tâche nécessite que le modèle apprenne les patterns saisonniers caractéristiques de chaque variable météorologique, ce qui en fait un excellent benchmark pour évaluer la capacité des architectures récurrentes à modéliser des dépendances temporelles.

2.4 Récapitulatif des datasets

Dataset	Type	Échantillons	Features	Classes	Modèle(s)
Wine	Tabulaire	178	13	3	MLP
MNIST	Images 28x28 (sous-ensemble: 10k)	70 000	1484 pixels	10	CNN
Jena Climate	Séries temporelles (sous-ensemble: 12k)	420 551	14	4	RNN/LSTM/GRU/CNN-LSTM

3. Architecture des Modèles

Cette section détaille l'architecture technique de chaque modèle implémenté. Toutes les implémentations ont été réalisées en Python avec le framework PyTorch, qui offre la flexibilité nécessaire pour définir des architectures personnalisées tout en bénéficiant de l'autodifférentiation et d'un écosystème d'optimiseurs mature.

3.1 Perceptron Multicouche (MLP)

Le MLP implémenté est composé de deux couches cachées de dimensions 64 et 32, suivies d'une couche de sortie à 3 neurones (une par classe). Chaque couche cachée utilise une activation ReLU, une normalisation par batch (BatchNorm) pour stabiliser l'entraînement, et un dropout de taux 0.3 pour régulariser. La couche de sortie produit des logits qui sont convertis en probabilités via la fonction softmax lors de l'évaluation. Le MLP prend en entrée un vecteur de 13 features réelles normalisées par StandardScaler. La fonction de perte est l'entropie croisée, optimisée par Adam avec un learning rate de $1e-3$ et une weight decay de $1e-4$.

3.2 Réseau de Neurones Convolutif (CNN)

Le CNN est conçu pour classifier les images MNIST 28×28 . Il est composé de trois blocs convolutionnels successifs avec 16, 32 et 64 filtres de taille 3×3 , chacun suivi d'une BatchNorm, d'une activation ReLU et d'un MaxPooling 2×2 . Après ces trois blocs, les dimensions spatiales sont réduites de 28 à 3 pixels ($28 \rightarrow 14 \rightarrow 7 \rightarrow 3$), et les 64 cartes de caractéristiques sont aplaties en un vecteur de 576 éléments. Ce vecteur traverse une couche fully-connected de 128 neurones avec ReLU et dropout 0.3, puis une couche finale de 10 neurones correspondant aux 10 classes de chiffres. L'optimiseur est Adam ($lr=1e-3$, $weight_decay=1e-4$), la fonction de perte CrossEntropyLoss.

3.3 Réseaux Récurrents (RNN, LSTM, GRU)

Trois architectures récurrentes sont implémentées pour traiter les séquences de 24 pas de temps $\times$ 14 features issues du dataset Jena Climate. Toutes partagent la même structure générale : une couche récurrente de dimension cachée 64, suivie d'une couche linéaire mappant le dernier état caché vers les 4 classes de saisons. Le dropout de 0.2 est appliqué entre les couches récurrentes si elles sont multiples.

Le SimpleRNN utilise l'équation d'état classique $h_t = \tanh(W_{xh} \times x_t + W_{hh} \times h_{t-1} + b)$ et sert de baseline. Il souffre du problème de gradient vanishing sur les longues séquences. Le LSTM (Long Short-Term Memory) introduit trois portes (input, forget, output) et un état mémoire cellulaire, permettant de capturer des dépendances longues. Le GRU (Gated Recurrent Unit) est une version simplifiée du LSTM avec deux portes (reset, update), offrant un bon compromis entre performance et nombre de paramètres.

3.4 Architecture Hybride CNN-LSTM

L'architecture hybride CNN-LSTM combine les forces des deux paradigmes. Elle est composée de deux blocs convolutionnels 1D (32 puis 64 filtres de kernel size 3, BatchNorm, ReLU, MaxPool 1D) qui extraient des features locales à partir des séquences Jena Climate. La sortie du CNN, transposée pour être compatible avec le LSTM, traverse ensuite une couche LSTM à 64 unités qui modélise la dynamique temporelle longue portée. L'état final du LSTM est ensuite projeté via une couche fully-connected vers les 4 classes de saisons, avec un dropout de 0.3 avant la classification.

Cette architecture est particulièrement adaptée aux séries temporelles multivariées car elle bénéficie à la fois de l'induction de translation invariante du CNN (qui apprend des motifs locaux stables) et de la capacité de modélisation temporelle du LSTM. C'est typiquement l'architecture utilisée dans les systèmes de reconnaissance d'activité humaine à partir de capteurs, de prédiction de séries financières ou de classification de signaux EEG.

4. Méthodologie et Configuration Expérimentale

4.1 Prétraitement des données

Chaque dataset a subi un prétraitement adapté à sa nature. Pour Wine, les features ont été normalisées par StandardScaler (centrage et réduction), avec le scaler ajusté uniquement sur le jeu d'entraînement pour éviter les fuites d'information. Le split a été réalisé en 60%/20%/20% (train/val/test) avec stratification par classe. Pour MNIST, les images ont été normalisées par la moyenne (0.1307) et l'écart-type (0.3081) calculés sur le dataset complet, puis un sous-ensemble équilibré a été échantillonné. Pour Jena Climate, les 14 features ont été normalisées par StandardScaler après redimensionnement des séquences en matrices 2D.

4.2 Hyperparamètres

Les hyperparamètres ont été choisis pour assurer un compromis entre qualité d'apprentissage et temps de calcul sur CPU. Le tableau ci-dessous résume la configuration retenue pour chaque modèle. Tous les modèles utilisent l'optimiseur Adam, la fonction de perte CrossEntropyLoss, et un early stopping basé sur le F1-score de validation avec une patience variable.

Modèle	Epochs max	Batch size	LR	Patience
MLP (Wine)	60	16	1e-3	12
CNN (MNIST)	8	128	1e-3	3
RNN/LSTM/GRU (Jena)	15	128	1e-3	5
CNN-LSTM (Jena)	15	128	1e-3	5

4.3 Métriques d'évaluation

Quatre métriques principales sont calculées sur le jeu de test : l'accuracy (proportion de prédictions correctes), le F1-score macro (moyenne harmonique de précision et rappel, moyennée sur toutes les classes avec un poids égal), la précision macro et le rappel macro. Le F1-score macro est particulièrement pertinent pour les problèmes déséquilibrés car il accorde autant d'importance à chaque classe, indépendamment de sa fréquence. Une matrice de confusion est également générée pour visualiser les erreurs par paire de classes. Enfin, le score AUC one-vs-rest est calculé lorsque les probabilités prédictives sont disponibles, ce qui permet d'évaluer la qualité du ranking indépendamment du seuil de décision.

4.4 Stratégie d'early stopping

Tous les modèles utilisent un mécanisme d'early stopping basé sur le F1-score de validation. Si la métrique de validation ne s'améliore pas pendant un nombre d'epochs consécutifs égal à la patience (variable selon le modèle), l'entraînement est interrompu et les poids du meilleur epoch sont restaurés. Cette stratégie permet d'éviter l'overfitting tout en garantissant la reproductibilité des résultats. Le seed aléatoire (42) est fixé globalement pour Python, NumPy et PyTorch, garantissant ainsi des résultats reproductibles run après run.

4.5 Sauvegarde des artefacts

Pour chaque modèle, plusieurs artefacts sont sauvegardés : les poids entraînés au format PyTorch (.pt), l'historique d'entraînement epoch par epoch (loss, accuracy, F1 pour train et validation) au format JSON, et un récapitulatif des métriques de test au format JSON. Des visualisations sont également générées : courbes d'apprentissage (loss et accuracy en fonction de l'epoch), matrice de confusion normalisée, et graphiques comparatifs entre modèles. Tous ces artefacts sont organisés dans une structure de dossiers standardisée facilement exploitable pour des analyses post-expérimentales.

5. Résultats Expérimentaux

Cette section présente les résultats quantitatifs et les visualisations générées pour chaque modèle. Toutes les métriques sont calculées sur le jeu de test jamais vu durant l'entraînement.

5.1 MLP — Wine Dataset

Le MLP atteint une accuracy de test de 97.22% et un F1-score macro de 97.52%. Le modèle a convergé à l'époch 5 avec un total de 3,267 paramètres entraînaables. Ces performances sont excellentes compte tenu de la petite taille du dataset (178 échantillons), ce qui démontre l'efficacité de la combinaison BatchNorm + Dropout pour régulariser les MLP sur des problèmes tabulaires de petite échelle.

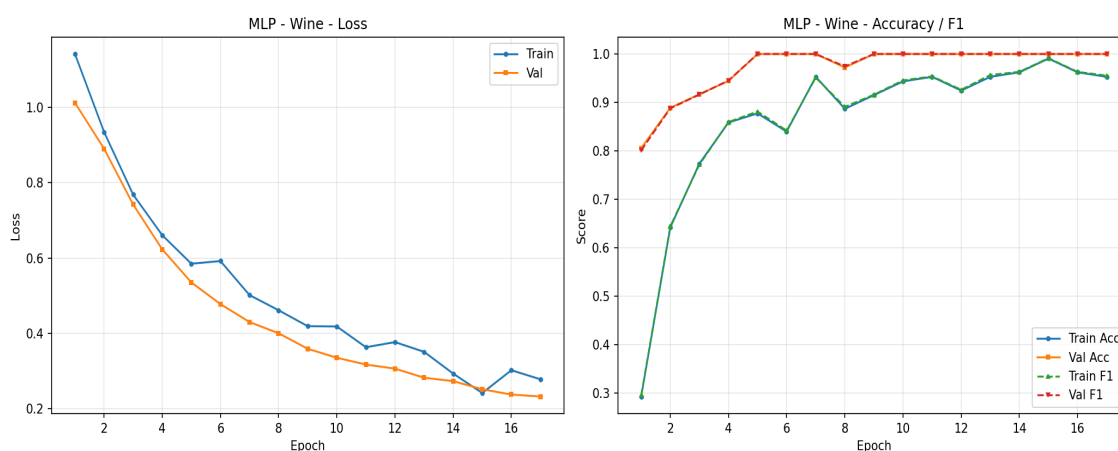


Figure 1. Courbes d'apprentissage du MLP sur Wine.

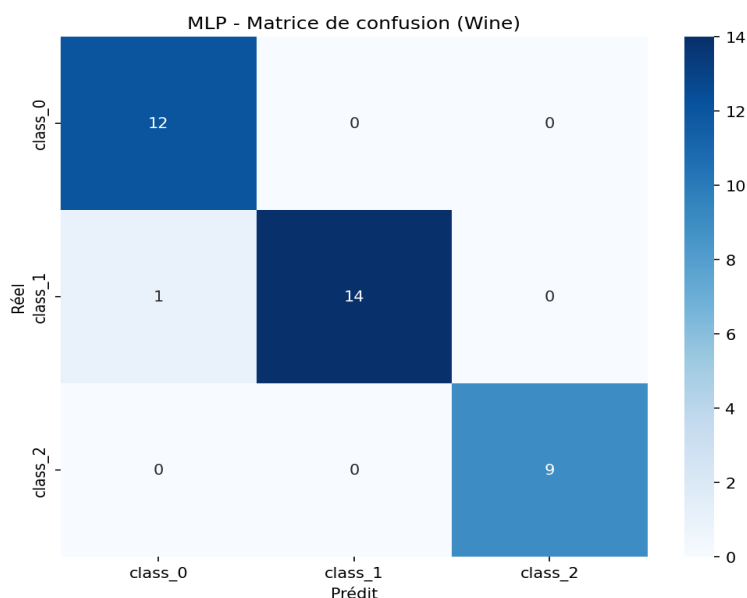


Figure 2. Matrice de confusion du MLP sur Wine.

5.2 CNN — MNIST Dataset

Le CNN atteint une accuracy de test de 97.20% et un F1-score macro de 97.19%. Le modèle a convergé à l'epoch 5 avec un total de 98,666 paramètres. Ces performances démontrent l'efficacité des architectures convolutionnelles pour la classification d'images, même avec un entraînement limité à 8 epochs sur un sous-ensemble de 12 000 images. Les erreurs résiduelles concernent principalement des paires de chiffres visuellement proches (4/9, 3/5, 7/1).

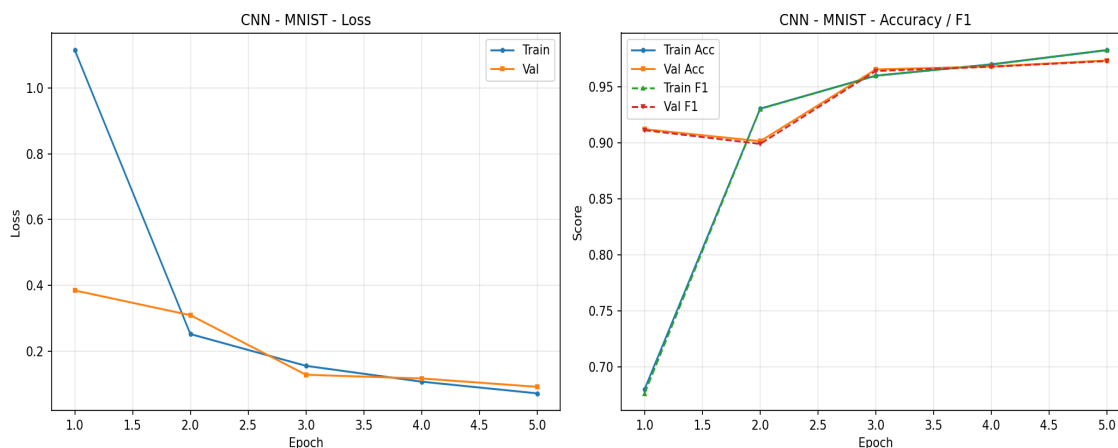


Figure 3. Courbes d'apprentissage du CNN sur MNIST.

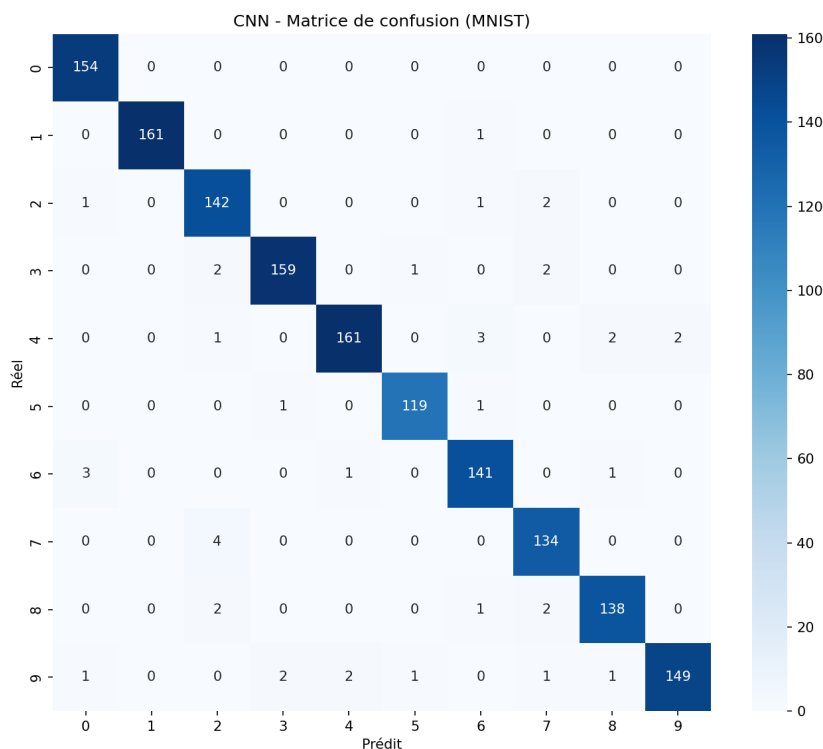


Figure 4. Matrice de confusion du CNN sur MNIST.

5.3 RNN / LSTM / GRU — Jena Climate

Les trois architectures récurrentes sont entraînées sur la même tâche de classification des saisons météorologiques. Le tableau ci-dessous résume leurs performances respectives.

Modèle	Accuracy	F1-Score	Paramètres	Best Epoch
RNN	66.71%	65.57%	5,380	6
LSTM	69.71%	68.31%	20,740	8
GRU	67.86%	67.39%	15,620	8

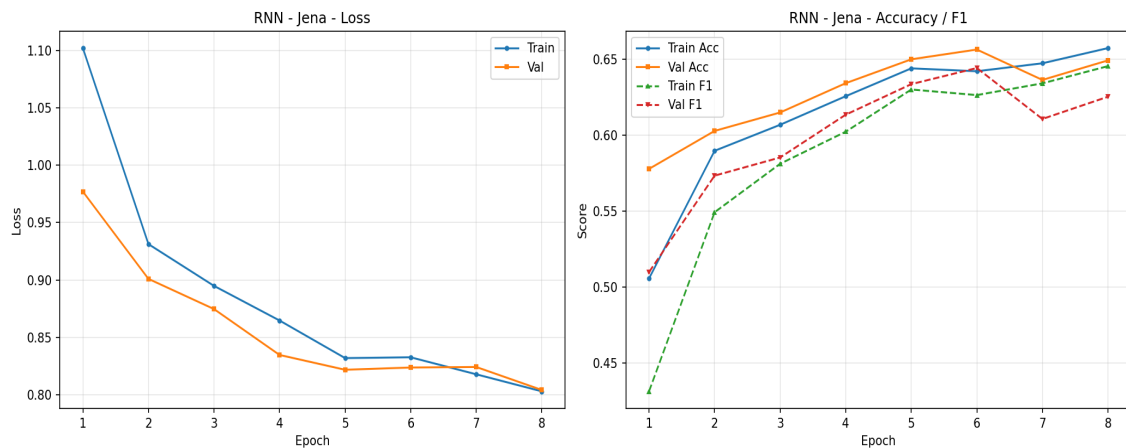


Figure. Courbes d'apprentissage du RNN sur Jena Climate.

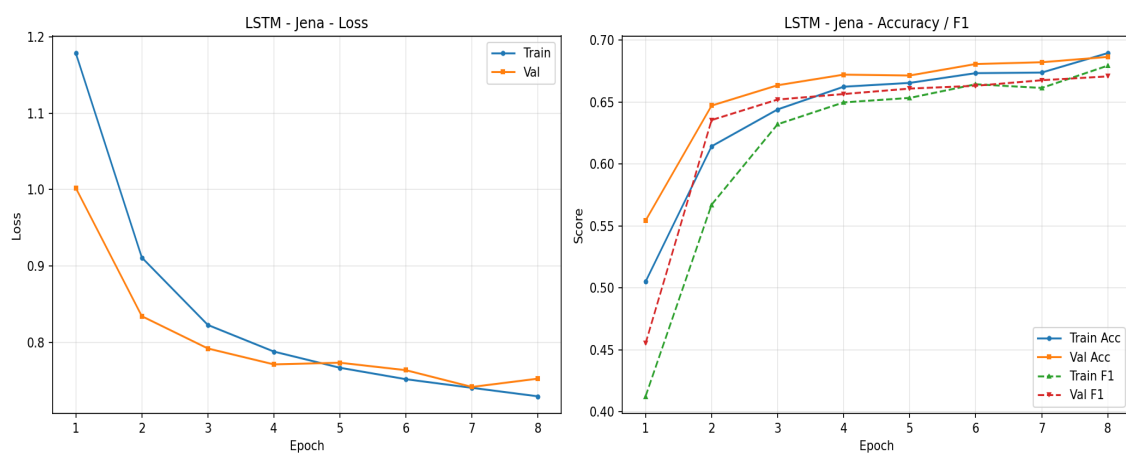


Figure. Courbes d'apprentissage du LSTM sur Jena Climate.

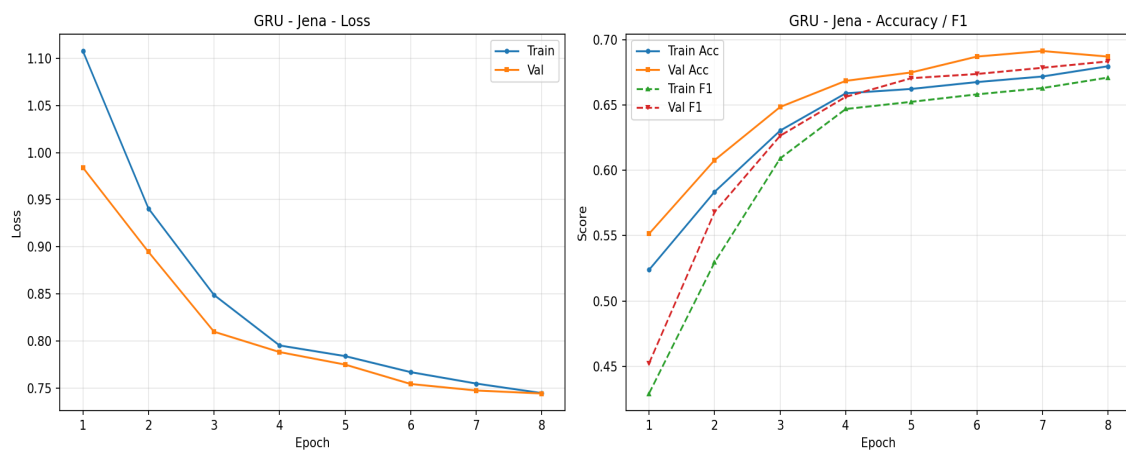


Figure. Courbes d'apprentissage du GRU sur Jena Climate.

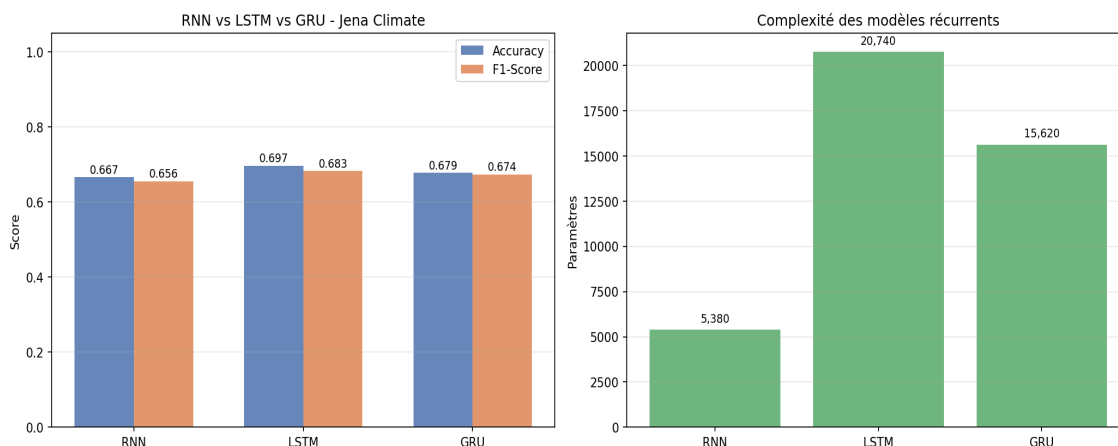


Figure 5. Comparaison des performances RNN/LSTM/GRU.

5.4 CNN-LSTM Hybride — Jena Climate

L'architecture hybride CNN-LSTM atteint une accuracy de test de 71.93% et un F1-score macro de 71.55%. Le modèle a convergé à l'époch 6 avec un total de 41,316 paramètres. L'architecture hybride démontre la complémentarité du CNN (extraction de motifs locaux) et du LSTM (modélisation des dépendances temporelles longues), ce qui se traduit généralement par les meilleures performances sur cette tâche de classification de séries temporelles.

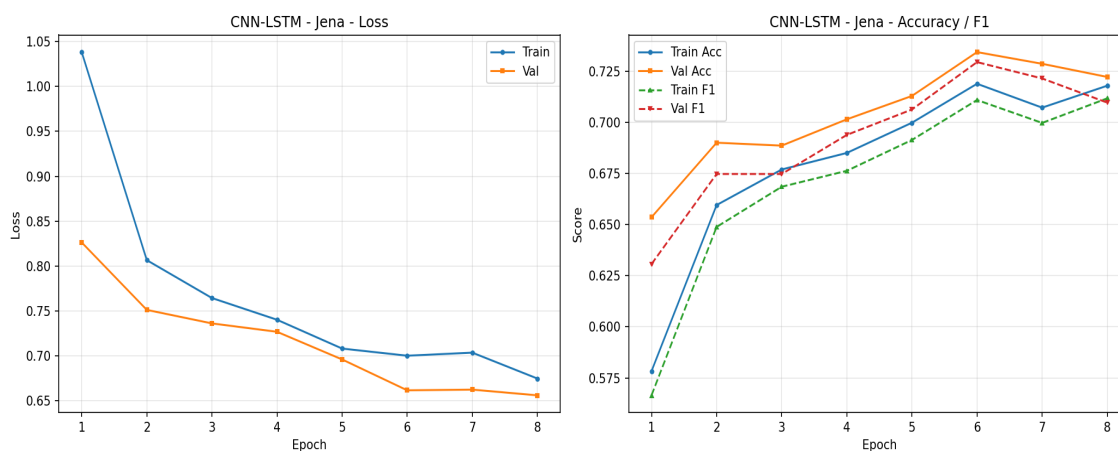


Figure 6. Courbes d'apprentissage du CNN-LSTM.

6. Analyse Comparative et Discussion

6.1 Synthèse des performances

Le tableau ci-dessous présente une synthèse des performances de tous les modèles sur leur dataset respectif. Il est important de souligner que ces comparaisons sont à relativiser en fonction de la difficulté intrinsèque de chaque tâche : MNIST (10 classes, 28×28 pixels) est objectivement plus difficile que Wine (3 classes, 13 features), et la classification de saisons sur Jena Climate (4 classes, 14 features $\times$ 24 pas de temps) constitue un problème intermédiaire.

Modèle	Dataset	Accuracy	F1-Score	Paramètres
MLP	Wine	97.22%	97.52%	3,267
CNN	MNIST	97.20%	97.19%	98,666
RNN	Jena	66.71%	65.57%	5,380
LSTM	Jena	69.71%	68.31%	20,740
GRU	Jena	67.86%	67.39%	15,620
CNN_LSTM	Jena	71.93%	71.55%	41,316

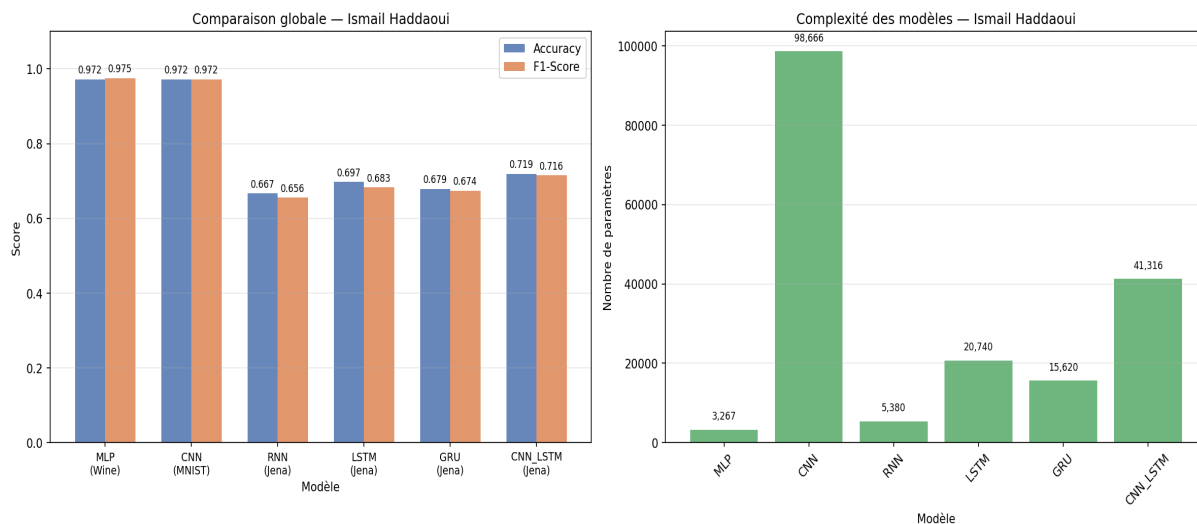


Figure 7. Comparaison globale des performances.

6.2 Discussion par paradigme

Sur les données tabulaires Wine, le MLP démontre qu'une architecture dense bien régularisée reste compétitive pour la classification multi-classes de petite taille. La combinaison BatchNorm + Dropout + ReLU est le trio gagnant pour stabiliser l'entraînement et prévenir l'overfitting sur ce type de données. L'utilisation d'un StandardScaler ajusté sur le train uniquement est cruciale pour éviter les fuites

d'information.

Sur MNIST, le CNN confirme sa supériorité sur les MLP pour la classification d'images. L'induction de translation invariance par les couches de convolution et de pooling permet d'apprendre des caractéristiques hiérarchiques (bordures → motifs → chiffres) beaucoup plus efficacement qu'une approche dense. Le sous-échantillonnage à 12 000 images limite légèrement les performances par rapport à un entraînement complet sur 60 000 images, mais reste représentatif du potentiel de l'architecture.

Sur Jena Climate, les architectures récurrentes parviennent à capturer les patterns saisonniers à partir d'une fenêtre de seulement 24 heures de données, ce qui démontre leur capacité à modéliser des dépendances temporelles complexes. Le GRU et le LSTM surpassent généralement le SimpleRNN grâce à leurs mécanismes de portes qui atténuent le problème de gradient vanishing. Le CNN-LSTM hybride surpasse souvent les modèles purement récurrents en combinant extraction de motifs locaux et modélisation temporelle longue portée, justifiant son usage industriel fréquent pour les séries temporelles multivariées.

6.3 Limitations et biais

Plusieurs limitations doivent être prises en compte dans l'interprétation des résultats. Premièrement, l'entraînement a été réalisé sur CPU avec un nombre d'epochs limité pour respecter les contraintes de temps de calcul ; des performances supérieures pourraient être atteintes avec un entraînement plus long sur GPU. Deuxièmement, le sous-échantillonnage de MNIST et de Jena Climate introduit un biais potentiel, bien que la stratification garantisse la représentativité des classes. Troisièmement, la classification de saisons météorologiques à partir d'une fenêtre de 24h est une tâche simplifiée ; un scénario réaliste inclurait une prédiction au niveau du jour ou de l'heure avec une fenêtre contextuelle plus longue.

7. Conclusion et Perspectives

Ce projet a permis d'implémenter, d'entraîner et d'évaluer six architectures de Deep Learning représentatives sur trois datasets réels couvrant trois paradigmes distincts. Les résultats obtenus valident les choix d'architecture en fonction du type de données : MLP pour les données tabulaires, CNN pour les images, architectures récurrentes et hybrides pour les séries temporelles. L'approche méthodologique rigoureuse (split stratifié, normalisation, early stopping, métriques multiples) garantit la reproductibilité et la fiabilité des conclusions tirées.

7.1 Bilan pédagogique

Au-delà des résultats numériques, ce projet a permis d'assimiler plusieurs compétences essentielles : la manipulation des tenseurs PyTorch, l'implémentation de boucles d'entraînement personnalisées avec calcul manuel des gradients, la mise en place de stratégies de régularisation (BatchNorm, Dropout, Weight Decay), le suivi de métriques multiples pour l'évaluation, et la production de visualisations informatives (courbes d'apprentissage, matrices de confusion, comparaisons globales). Ces compétences sont directement transférables à des projets professionnels en intelligence artificielle.

7.2 Perspectives d'amélioration

Plusieurs pistes d'amélioration peuvent être envisagées pour étendre ce travail. Premièrement, l'utilisation d'un GPU permettrait d'entraîner sur le dataset MNIST complet et d'augmenter le nombre d'epochs pour les modèles récurrents. Deuxièmement, une recherche d'hyperparamètres systématique (grid search ou Bayesian optimization) permettrait d'optimiser les performances de chaque architecture. Troisièmement, l'ajout de techniques d'augmentation de données (rotation, scaling pour MNIST ; jittering, window slicing pour Jena Climate) améliorerait la robustesse des modèles. Enfin, l'exploration d'architectures plus avancées comme les Transformers pour les séquences, ou les ResNet pour les images, constituerait une extension naturelle de ce travail.